

CMR SPECIALIST · PHASE 2 (v3)

The Tool Module: Formulator & Firer

A sealed database-access layer for the CMR Specialist. The module formulates and fires every database call; the harness selects and fills. Two verbs only — CHECK for verification, WRITE for direct database writes — against a mock database the harness and LLM cannot touch.

Item	Detail
Module role	FORMULATOR + FIRER — owns the catalog of call templates; sole executor against the DB
Harness role	SELECTOR + FILLER — reasons over complaints, picks a formulation, fills it with case facts
Verbs	CHECK (verification tasks) · WRITE (direct DB writes). Nothing else exists.
Target	Mock CMR database today; real CMR database tomorrow — same contracts, zero harness changes
Memory	Module owns an independent audit log; case history stays in the case DB

DESIGN RULE. If a call is not formulated in the catalog, it cannot be made. There is no such thing as an off-catalog database action.

1 · The core idea: a sealed doorway

The harness and the LLM hold **no database credentials and no connection strings** — not to the mock, not to anything. The tool module holds them all. Data can only be reached through the module's formulated API calls, and every call is validated, authorized, executed, and logged by the module itself.



Figure 1 — the only path to data. The harness never crosses the doorway; the module never reasons about complaints.

Division of labour, fixed permanently:

	Module (formulator + firer)	Harness (selector + filler)
Owens	Catalog templates, credentials, connections, audit log	Complaint reasoning, formulation choice, case facts
Does	Validate filled forms, fire calls, enforce approvals + idempotency, log everything	Pick formulation, fill blanks, present approvals, assemble cited answers
Never	Reasons about complaints, invents calls, self-authorizes writes	Fires calls, holds DB credentials, touches the DB directly

2 · Two verbs: CHECK and WRITE

CHECK is for verification tasks. It looks at the database and answers — both plain lookups (*does this profile exist?*) and verdicts (*does this NIN match? is this payment on the right account?*). Verdicts live *inside* check results as a citable field, not as a separate operation. Checks are read-only, freely callable, never approved.

WRITE writes directly to the database. Creates, updates, links, resends. Every write carries three attachments: a human approval, an idempotency key, and — where applicable — the evidence ref of the check that justified it. No exceptions, no self-authorization.

	CHECK	WRITE
Purpose	Verification tasks: look up + judge	Direct DB writes: create / update / link / resend
Side effects	None, ever	Always; each one named and logged
Approval	Never needed	Always required (single; dual for flag-class writes if added)
Idempotency	Not required (safe to repeat)	Mandatory key; repeats are ignored, never re-applied
Failure mode	Returns not-found / unverifiable	Returns error_ref; never claims success

NOTE. Adding a capability means adding one formulation to the catalog — endpoint, schema, permission, approval, idempotency — with zero changes to harness code. The catalog is the only file that grows.

3 · Anatomy of a formulation

Every template in the catalog — check or write — is specified with exactly these fields. The formulation *is* the policy: read one template and you know everything about how that call behaves.

```
CHECK.receipt.lookup:
verb: CHECK
endpoint: GET /tool/check/receipt (module API, not the DB)
inputs: { remita_rrr: string(12-digit, required) }
permission: standard approval: none
idempotency: not-required timeout: 15s retries: 2 (reads only)
returns: { status, amount, date, linked_account, verdict }
```

```
WRITE.payment.link:
verb: WRITE
endpoint: POST /tool/write/payment-link
inputs: { remita_rrr: required, account_identifier: required }
permission: sensitive approval: single-operator
idempotency: required timeout: 60s retries: 0 (never blind-retry a write)
evidence: requires CHECK.receipt.lookup verdict=paid on same run
returns: { link_ref, status } or { error_ref }
```

Note the last line of the write template: a write formulation *names the check verdict it depends on*.

Verify-before-write is therefore not a guideline — it is a field the firing pipeline enforces.

4 · CHECK catalog (verification tasks)

Formulation	Answers question	Key inputs → verdict
CHECK.profile.lookup	Does this applicant have a profile? (J1 OTP)	phone/email → exists, username type
CHECK.nin.verify	Does the NIN match / is it valid? (J2)	NIN → match / mismatch / suspended

Formulation	Answers question	Key inputs → verdict
CHECK.nimc.health	Is the NIMC portal actually down? (J2)	— → up / degraded / down (sole downtime source)
CHECK.vehicle.lookup	What is on record for this vehicle? (J3–J5)	plate/chassis → record, cert state
CHECK.owner.lookup	Who legally owns it? (J3)	vehicle → owner + history count
CHECK.buyer.search	Is the buyer known? (J3)	phone/email/NIN/TIN → profile match
CHECK.receipt.lookup	What is this payment's state? (J4)	RRR → status, amount, date, linked account
CHECK.certificate.lookup	What is this certificate's state? (J5)	plate → processing/approved/expired, age

DND-status diagnosis (text ALLOW / STATUS to 2442) and the password-reset walkthrough stay grounded knowledge answers — no database is involved, so no formulation exists for them. OCR of receipt photos and transfer documents is an *upstream* helper (it reads images, not the DB) whose output feeds check inputs.

5 · WRITE catalog (direct DB writes)

Formulation	Writes what (to mock now, CMR later)	Gate
WRITE.token.resend	Password-reset token via phone/email (J1)	Single approval
WRITE.transfer.initiate	Ownership transfer, seller-initiated (J3)	Single approval; seller-only rule
WRITE.payment.confirm	Confirm-payment action on a request (J4)	Single approval
WRITE.payment.link	Link RRR to correct account (J4)	Single approval + receipt=paid evidence
WRITE.certificate.resend	Re-send certificate to email (J5)	Single approval
WRITE.certificate.renew	Renew expired certificate (J5)	Single approval + paid-RRR evidence
WRITE.certificate.correct	Field correction with reason (J5)	Single approval + window rule + value allowlist

Out of the catalog by design: stolen-flag unflagging (SOP investigation chain stays human), bank-side reversals (guidance + escalation only), and notifications (downstream sender, not a DB operation). The doorway stays a doorway.

WARNING. Open NPF item carried forward: call-centre scripts allow certificate changes up to 3 months, the SOP refers post-30-day corrections to HQ. WRITE.certificate.correct ships with the stricter 30-day default until ruled.

6 · The firing pipeline (inside the module)

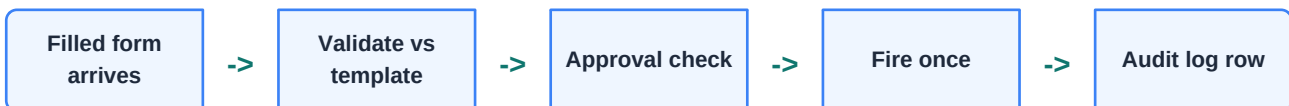


Figure 2 — every call passes all five stages. A failure at any stage stops the call and is itself logged.

Stage	What happens	On failure
1 · Validate	Form matched to catalog template; types, required fields, value rules checked	Rejected with field-level reason; harness asked to refill
2 · Authorize	CHECK: pass. WRITE: valid approval present? evidence ref present and fresh?	WRITE parks in pending_approval; missing evidence → refused with reason
3 · Fire	Single DB execution via the repository; writes never auto-retried	Transient error → bounded retry (reads) or terminal error_ref (writes)
4 · Respond	Typed envelope: {status, data, evidence_refs} or {status: failed, error_ref}	Envelope always well-formed — callers never parse raw DB errors

Stage	What happens	On failure
5 · Audit	Row appended to module audit log (next section) before the response returns	If the audit write fails, the call is treated as failed — no silent actions

7 · The module's own audit system

Two logs exist because two different questions get asked. The **case log** (harness side) answers *what did the agent do for this citizen?* The **module audit log** answers *what happened to the data?* When the two disagree, the module log is ground truth — which is exactly why it must belong to the module, not the harness.

Audit field	Recorded	Why
audit_id / timestamp	UUID + UTC time	Ordering and tamper-evident sequence
formulation_id + verb	e.g. WRITE.payment.link	Which template fired
caller ref	Harness run_id + case_id	Joins module truth to case story
arguments (PII-masked)	NIN/RRR last-4 only	Audit without leaking identity data
result summary	Status + refs (link_ref, verdict)	What changed / was found
evidence refs	Check verdict IDs cited	Proves verify-before-write after the fact
approval ref	Approver + time (writes)	Who authorized the mutation
latency + error	ms + error_ref if failed	Performance and failure analysis

DESIGN RULE. The audit log is append-only. Nothing — not even an operator — edits or deletes rows. Corrections are new compensating entries, exactly like case-field versioning.

8 · Integration with the harness

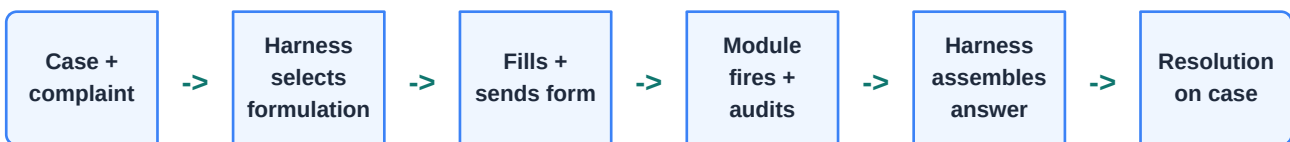


Figure 3 — integration flow. The harness plans in formulations the way it already plans in tools; the worker, queue, approvals, and citations are v2.0 as built.

Seam (v2.0, exists)	What changes	Size
Planner task flows	Plan steps name formulations (CHECK.*, WRITE.*) per job family J1–J5	Config + tests
Policy gate	Only two rules to enforce: checks pass freely; writes need approval + evidence + key	Simpler than before
Registry / runner	Runner becomes a thin client of the module API (service token, envelope in/out)	Small
Approvals UI	Approve/reject already exists; now explicitly releases a WRITE formulation	None
run_steps + citations	Steps record formulation_id + evidence refs; check verdicts cited like chunks	None
Worker + queue	Unchanged — leases, retries, resumes formulation calls like any step	None

What crosses the boundary, in either direction: filled forms and typed envelopes out; results and evidence refs back. What never crosses: credentials, connection strings, SQL, raw DB errors, full NINs.

9 · Mock database + conformance suite

The mock is a conformance stand-in, not a shortcut: deterministic seed data (profiles, vehicles, receipts, certificates, flags) behind a repository interface, with error injection (timeouts, 404s, downtime flags) so retry, fallback, and escalation paths are testable from day one.

The conformance suite is the exam paper with an answer key that any backend — mock today, real CMR tomorrow — must pass identically before cutover:

#	Exam question	Must answer
1	CHECK.receipt.lookup on known RRR	status=paid + correct amount, date, account
2	CHECK.vehicle.lookup on unknown plate	not-found envelope — never a crash, never invented data
3	WRITE.payment.link twice, same key	One link; second call ignored with duplicate-key notice
4	WRITE without approval	Refused with reason; zero DB effect, audit row still written
5	WRITE citing stale/missing evidence	Refused; names the missing check verdict
6	Backend timeout on a CHECK	Retryable error inside 5s; harness may retry

DESIGN RULE. Cutover rule: the real CMR backend must score identically on the full suite before a single citizen case touches it. Write the suite now, while the mock is being built.

10 · Worked example: paid-but-unpaid (J4)

#	Side	Action	Stored
1	Harness	Select CHECK.receipt.lookup, fill RRR	Proposal + filled form
2	Module	Fire check; verdict: paid, wrong account	Audit row + evidence ref E-101
3	Harness	Select WRITE.payment.link, fill RRR + account, cite E-101	Approval requested
4	Operator	Approve	Approval ref A-77; task released
5	Module	Fire write once (key K-55); re-check confirms link	Audit rows; link_ref L-8812
6	Harness	Assemble cited resolution	Task completed on case

If the worker restarts after step 5, replay finds key K-55 completed and continues — the payment is never linked twice.

Safety rails + build order

- **Two verbs only.** Anything that is not a check or a write does not enter the catalog — OCR stays upstream, notifications downstream, doorway stays a doorway.
- **Writes never self-authorize, never blind-retry, never claim unverified success.**
- **Audit is append-only;** corrections are compensating entries.
- **Phase 2a:** module shell + mock + conformance suite + all 8 CHECKS → exit: complaints answered with live mock facts + citations; suite green.
- **Phase 2b:** first writes (WRITE.payment.link, WRITE.certificate.renew) → exit: one payment and one renewal fixed end to end, zero duplicates, approvals honored.
- **Phase 2c:** remaining writes + correction window ruling → exit: all five families pass safety tests; NPF confirms window rule and endpoint shapes for the real cutover.